Disciplina: FACOM31701 - Trabalho de Conclusão de Curso 1

Professor: Prof. Dr. Rafael Dias Araújo

Estudante: Rafael Lopes Fraga

Orientador(a): Prof. Dr. Paulo Rodolfo da Silva Leite Coelho

Tipo de trabalho: Projeto de Pesquisa Tradicional

Entrega 2 (E2) - Visão geral sobre a fundamentação teórica

1) Identifique, juntamente com o seu orientador(a), quais são os principais conceitos teóricos associados ao seu trabalho. Exemplo 1: se o tema do trabalho é o uso de algoritmos genéticos para análise de risco de crédito, então os principais conceitos são: algoritmos genéticos e análise de risco de crédito. Exemplo 2: se o tema do trabalho é o desenvolvimento de um chatbot para atender a comunidade UFU, então os principais conceitos são: processamento de linguagem natural, chatbots e formas de atendimento à comunidade comumente feitas pelas universidades.

2) Escreva sobre cada um dos conceitos identificados no item anterior. Use a seguinte estratégia: i) forneça as principais definições; ii) apresente um breve histórico; iii) discuta os avanços recentes da área; e iv) mostre alguns exemplos relevantes e ilustrativos. Para todos os casos, incluir as referências consultadas. O objetivo aqui é ter um rascunho da Seção 2 - Fundamentação Teórica. Portanto, é de suma importância analisar outras monografias para entender o que deve ou não ser escrito nesta seção. Outras referências bibliográficas também poderão ser consultadas (por exemplo, artigos científicos e livros).

3) Ao final desta entrega, declare o uso ou não de Inteligência Artificial para produção deste trabalho.

Observações:

1. No caso do **relatório técnico profissional**, o texto a ser apresentado será i) um detalhamento do ambiente de atuação (empresa/instituição onde se trabalha), segmento de atuação, quais são as suas atribuições e responsabilidades dentro da empresa; ii) a definição dos conhecimentos necessários para a execução do trabalho e as restrições técnicas e operacionais do ambiente; iii) os padrões e boas práticas adotados pela equipe para realização das atividades profissionais; iv) o projeto que irá desenvolver destacando seu enquadramento e relevância no contexto empresarial/institucional e acadêmico. Para este modelo, deverá ser entregue também um termo de ciência e

concordância para desenvolvimento do trabalho, assinado pelo representante legal e/ou chefia imediata na empresa onde trabalha (ver modelo anexado à entrega).

2. No caso do **desenvolvimento de um software**, crie seções para i) estudo preliminar sobre a viabilidade da proposta, incluindo relevância, diferenciais e motivações, ii) as tecnologias específicas que se pretende utilizar com breve apresentação , histórico e justificativas para a escolha, iii) os principais princípios, técnicas e conceitos de engenharia de software a serem aplicados no desenvolvimento; iv) seções descritivas relacionadas ao domínio/subdomínios envolvidos na aplicação e necessárias para o entendimento da solução.

3. No caso de um **plano de negócios**, descreva: i) o domínio da aplicação e negócio a ser abordado; ii) uma versão inicial do produto e/ou serviço que será implementado no plano de negócios contendo objetivos, motivações e características; iii) benefícios, diferenciais, limitações e custos envolvidos; iv) o estágio atual de desenvolvimento, expectativas e planejamento preliminar.

4. É de suma importância que todos os parágrafos escritos sejam coesos. Ou seja, o texto como um todo precisa fazer sentido;

5. Use o formato de citações do Latex para apresentar as referências em todo o texto. Deve ser entendido como as citações funcionam no Latex, em particular com o BibTeX.

6. O posicionamento de citações no texto e orientações da ABNT serão discutidos nas aulas. Lembre-se que no modelo fornecido para a entrega 1, além do arquivo `main.tex` utilizado para redigir seu texto, há um arquivo chamado `ref.bib` com alguns exemplos de referências. Um tutorial sobre o overleaf está disponível em (Overleaf, 2025).

7. Junto da entrega, o aluno deverá anexar um comprovante de concordância do seu orientador quanto ao texto sendo submetido. O comprovante de concordância poderá ser um e-mail do seu orientador, onde ele manifesta autorização para submeter o texto, ou uma assinatura eletrônica no PDF submetido feita pelo orientador, preferencialmente pelo sistema GovBr (ou outra plataforma de assinaturas conhecida e com validação pública).

1 RASCUNHO DA FUNDAMENTAÇÃO TEÓRICA

Tema do trabalho: Padrão Saga em Arquiteturas de Microsserviços: Um Estudo Comparativo entre os Modelos Coreografado e Orquestrado.

A fundamentação abrange arquitetura de microsserviços, transações locais e distribuídas, modelos de consistência, Sagas e comunicação orientada a mensagens. Esses conceitos permitem examinar como componentes autônomos cooperam e quais garantias podem ser oferecidas quando uma operação envolve dados mantidos em diferentes processos.

1.1 Arquitetura de microsserviços

1.1.1 DEFINIÇÃO, MODULARIDADE E AUTONOMIA

Tanenbaum e Van Steen (2007, seção 1.1) definem um sistema distribuído como uma coleção de computadores independentes que se apresenta aos usuários como um sistema único e coerente. A comunicação por rede introduz latência e falhas parciais: um processo pode continuar ativo enquanto outro falha, e uma solicitação pode ser executada mesmo que sua resposta se perca. A transparência da distribuição procura ocultar a localização e a organização dos recursos das aplicações que os utilizam (Van Steen; TANENBAUM, 2023, p. 10–13).

Os microsserviços organizam uma aplicação como um conjunto de serviços que executam em processos separados e se comunicam apenas por interfaces. Van Steen e Tanenbaum (2023, p. 64–65) tratam o estilo como uma forma de arquitetura orientada a serviços cujo princípio central é a modularização: cada serviço encapsula sua implementação e expõe aos demais somente as operações que oferece. Por conseguinte, a interface permite que os clientes utilizem o serviço sem conhecer sua organização interna, que pode ser modificada desde que sejam preservados as operações oferecidas e seu comportamento esperado.

Lewis e Fowler (2014) caracterizam microsserviços como serviços implantáveis de maneira independente e organizados em torno de capacidades de negócio, de modo que cada serviço reúne a lógica e as camadas técnicas necessárias para desempenhar uma função, como administrar um catálogo ou processar pagamentos.

A implantação independente depende da estabilidade das interfaces. Quando uma

alteração exige modificações simultâneas em vários serviços, os ciclos de entrega ficam acoplados. O limite de um microsserviço deve reunir responsabilidades coesas e encapsular as decisões internas que podem mudar durante sua evolução (LEWIS; FOWLER, 2014).

O gerenciamento dos dados acompanha essa delimitação. No padrão *Database per Service*, cada serviço controla seus dados persistentes, e os demais os acessam por sua interface. A separação estabelece a responsabilidade sobre os dados, admitindo infraestrutura física compartilhada. A privacidade dos dados reduz dependências sobre esquemas internos, mas faz com que operações envolvendo vários serviços precisem de mecanismos de coordenação (RICHARDSON, 2018, cap. 2 e 4).

1.1.2 ANTECEDENTES E DESENVOLVIMENTO DA ARQUITETURA

O encapsulamento de objetos e as arquiteturas orientadas a serviços antecedem os microsserviços. A separação entre interface e implementação permite combinar componentes com responsabilidades próprias. Na exposição de Van Steen e Tanenbaum (2023, p. 64–66), essa organização se estende a serviços executados em processos distintos e retoma a composição de programas pequenos da tradição Unix.

Em 2014, Lewis e Fowler (2014) reuniram características que vinham aparecendo em aplicações organizadas dessa forma, entre elas implantação independente, descentralização dos dados e automação da infraestrutura. Posteriormente, Richardson (2018) sistematizou padrões para decomposição, comunicação e transações, explicitando dificuldades que surgem quando funções antes executadas em um único processo passam a depender da cooperação entre serviços.

A distribuição dos componentes também se ampliou. Além dos centros de dados, serviços passaram a executar em infraestruturas de borda, próximas dos usuários, de modo que a localização de um serviço influencia a latência de comunicação e os recursos disponíveis para executá-lo. A decisão arquitetural envolve, assim, os limites lógicos dos serviços e seu posicionamento físico (Van Steen; TANENBAUM, 2023, cap. 2).

Em uma loja virtual, a aplicação combina as informações fornecidas pelo catálogo, como descrições e preços, com a autorização de cobranças pelo serviço de pagamentos para realizar uma compra que depende da atuação de ambos. Como cada serviço controla sua implementação e seus dados, uma operação que modifica recursos administrados separadamente exige definir quais alterações devem compor a mesma unidade transacional (Van Steen; TANENBAUM, 2023, p. 65–66).

1.2 Transações e confirmação distribuída

1.2.1 TRANSAÇÕES EM BANCOS DE DADOS E PROPRIEDADES ACID

Uma transferência bancária exige que débito e crédito pertençam à mesma unidade de trabalho. Gray (1981) caracteriza a transação como uma transformação de estado sujeita a atomicidade, consistência e durabilidade. Em um banco de dados, essa unidade reúne operações de leitura e escrita e termina com confirmação, ou *commit*, ou aborto. Sendo assim, a confirmação torna seus efeitos duráveis, o que faz o aborto desfazer as alterações da tentativa.

Para representar uma transação simples, sejam $r_T(x)$ a leitura do item x pela transação T, $w_T(x, v)$ a escrita do valor v, c_T sua confirmação e a_T seu aborto. Uma transferência de valor v entre contas com saldos A e B pode ser descrita pela sequência

$$T = \langle r_T(A),\ r_T(B),\ w_T(A, A - v),\ w_T(B, B + v),\ c_T \rangle.$$

As leituras fornecem os saldos empregados nas escritas. Sob a hipótese de ausência de tarifas, a operação preserva $A + B$ e verifica as restrições aplicáveis, como saldo suficiente. Uma falha antes da confirmação encerra a tentativa com a_T e exige desfazer as alterações locais (GRAY, 1981).

Às propriedades descritas por Gray, Haerder e Reuter (1983) acrescentam o isolamento e fixam a sigla ACID para os requisitos de execução e recuperação de transações. Na transferência bancária, a *atomicidade* impede que o débito seja confirmado sem o crédito: ou as duas escritas valem, ou nenhuma. A *consistência* exige que a transação leve o banco de um estado que satisfaz suas restrições de integridade a outro que também as satisfaz e, assim, parte dessas restrições é declarada no banco e parte depende da correção da própria lógica da operação. O *isolamento* impede que transações concorrentes interfiram entre si, de modo que uma consulta não veja um saldo já debitado cujo crédito ainda não ocorreu. A *durabilidade* garante que, depois do *commit*, o resultado sobreviva a uma queda do processo ou do equipamento (Van Steen; TANENBAUM, 2023, p. 37–39).

A execução concorrente pode intercalar operações de transações distintas. Para explicitar essa ordem, seja T_i uma sequência de leituras e escritas terminada por c_i ou a_i. Uma história completa H intercala essas sequências, preservando a ordem interna de cada T_i (BERNSTEIN; HADZILACOS; GOODMAN, 1987). Uma história serial executa cada transação integralmente antes da seguinte. A equivalência com uma história serial fornece o critério de correção para a intercalação (ESWARAN et al., 1976).

Duas operações conflitam quando pertencem a transações diferentes, acessam o mesmo item e pelo menos uma delas é uma escrita. Para o conjunto de transações confirmadas, uma história é serializável por conflitos se existe uma história serial que preserva a ordem de todos esses pares. Por exemplo, $r_1(x), w_1(x), r_2(x), w_2(x), c_1, c_2$ mantém

os conflitos de T_1 antes dos de T_2 e corresponde à ordem serial T_1, T_2. A serializabilidade por conflitos é o critério adotado nesta formalização (BERNSTEIN; HADZILACOS; GOODMAN, 1987).

Os níveis de isolamento determinam quais interferências são admitidas. Berenson et al. (1995) analisam as ambiguidades da caracterização desses níveis por leituras sujas, leituras não repetíveis e fantasmas, e definem o isolamento por instantâneo, ou *snapshot isolation*. Nesse último, transações concorrentes podem tomar decisões sobre a mesma versão dos dados e produzir uma execução não serializável. Assim, a garantia de isolamento deve ser especificada separadamente da atomicidade.

O controle de concorrência e a recuperação se consolidaram nos bancos multiusuários. Em 1976, Eswaran et al. (1976) relacionaram consistência, escalonamento de operações e bloqueios; Gray (1978) reuniu mecanismos de execução e recuperação; Haerder e Reuter (1983) sistematizaram as propriedades ACID. A distribuição dos dados acrescentou a esses mecanismos a necessidade de coordenar decisões entre gerenciadores.

1.2.2 TRANSAÇÕES DISTRIBUÍDAS

Uma transação local opera sob o controle de um gerenciador de recursos. A transação distribuída envolve recursos administrados por diferentes processos, cujas partes locais devem respeitar uma decisão comum. Esse problema ocorre em bancos distribuídos quando uma operação alcança dados de partições distintas. No CockroachDB, por exemplo, transações podem acessar múltiplas unidades de distribuição denominadas *ranges*, exigindo coordenação entre os nós responsáveis (TAFT et al., 2020).

Os processos responsáveis pelas partes locais de uma transação distribuída são chamados de participantes e informam ao coordenador se possuem condições de confirmar suas operações. Com base nessas informações, o coordenador conduz o protocolo de confirmação e comunica a decisão global, que depende da concordância de todos os participantes para que, assim, a transação seja confirmada (GRAY, 1978).

A confirmação atômica distribuída exige uma decisão final comum: todos os participantes confirmam suas partes ou todos as abortam. A confirmação depende da concordância de todos os participantes. Uma falha de comunicação torna essa decisão mais difícil, pois uma resposta ausente pode decorrer tanto de falha quanto de atraso. O protocolo precisa preservar a decisão mesmo nessas situações (Van Steen; TANENBAUM, 2023, seção 8.5).

1.2.3 PROTOCOLO DE CONFIRMAÇÃO EM DUAS FASES

O protocolo de confirmação em duas fases, ou *Two-Phase Commit* (2PC), separa a preparação dos participantes da decisão global. Na primeira fase, o coordenador solicita os votos. Cada participante verifica se pode confirmar e registra de forma persistente o

estado necessário à recuperação antes de votar favoravelmente. A partir desse voto, deve conservar as condições para cumprir a decisão global (GRAY, 1978).

Na segunda fase, o coordenador decide pela confirmação somente se todos os votos forem favoráveis. Uma rejeição ou o término do prazo de coleta sem todos os votos favoráveis permite decidir pelo aborto. A decisão é registrada em armazenamento persistente e comunicada aos participantes, que a aplicam localmente. Os registros de recuperação permitem retomar o protocolo e retransmitir a decisão após uma interrupção (GRAY, 1978; LAMPSON; STURGIS, 1979).

Na transferência entre dois bancos, basta que um deles vote contra, por exemplo por saldo insuficiente, para que o coordenador decida abortar e nenhum altere seus dados. Se ambos votam a favor e o coordenador registra a confirmação, cada banco aplica sua parte, ainda que em instantes diferentes ou só depois de se recuperar de uma falha (Van Steen; TANENBAUM, 2023, p. 529–533).

O ponto crítico do protocolo está entre os dois votos. Suponha que o coordenador decida confirmar, comunique a decisão a um dos bancos e falhe antes de alcançar o outro. O segundo banco já votou a favor e não sabe o que foi decidido: abortar por conta própria seria incompatível com a confirmação já aplicada pelo primeiro, e confirmar seria incompatível com um eventual aborto. Ele precisa aguardar que o coordenador se recupere ou que algum processo acessível conheça a decisão. É essa espera que torna o 2PC um protocolo bloqueante (Van Steen; TANENBAUM, 2023, p. 530–531).

O tratamento de falhas durante a confirmação aparece em Gray (1978) e no relatório de Lampson e Sturgis (1979). Este último examina atualizações distribuídas sobre várias máquinas e explicita as hipóteses de armazenamento, processamento e comunicação necessárias à recuperação. O registro persistente permite que processos reiniciados recuperem os compromissos assumidos antes da interrupção.

A replicação geográfica aumenta o custo das rodadas de comunicação necessárias à confirmação. Taft et al. (2020) apresentam o *Parallel Commits*, que sobrepõe a replicação de um estado intermediário da transação à verificação das escritas pendentes. Quando ambas terminam, o coordenador pode reconhecer a confirmação sem aguardar uma rodada sequencial adicional de consenso, mantendo a decisão atômica.

1.3 Modelos de consistência em sistemas distribuídos

1.3.1 CONSISTÊNCIA DOS DADOS E CONSISTÊNCIA TRANSACIONAL

Um modelo de consistência estabelece quais resultados podem ser observados quando processos leem e modificam dados compartilhados. Em um armazenamento replicado, diferentes cópias de um mesmo item podem receber atualizações em momentos distintos. O

modelo restringe os valores retornados pelas leituras e as ordens de operações consideradas admissíveis. Van Steen e Tanenbaum (2023, p. 395–396) descrevem essa relação como um contrato entre os processos e o armazenamento: as garantias dependem das regras que ambos devem cumprir.

A consistência de ACID exige preservar as restrições de integridade do banco (HAERDER; REUTER, 1983). Em dados replicados, o modelo de consistência determina quais atualizações uma leitura pode observar: um cliente que lê de réplicas distintas pode obter uma versão anterior de um item, ainda que cada réplica satisfaça suas restrições locais (TERRY et al., 1994).

1.3.2 CONSISTÊNCIA FORTE E ORDENAÇÃO DAS OPERAÇÕES

A expressão *consistência forte* é usada para garantias que aproximam o comportamento do armazenamento distribuído do acesso a uma única cópia. Uma formulação precisa é a *linearizabilidade*, definida por Herlihy e Wing (1990): cada operação deve parecer produzir seu efeito em um único instante entre sua invocação e sua resposta, respeitando a especificação do objeto. Além disso, a ordenação precisa respeitar a precedência em tempo real entre operações que não se sobrepõem.

Se uma escrita de 1 em um registrador x termina antes do início de uma leitura, sem outra escrita, a leitura deve devolver 1. Operações sobrepostas podem admitir diferentes ordens. As réplicas devem responder de acordo com essa ordem observável (HERLIHY; WING, 1990).

Lamport (1979) define consistência sequencial pela existência de uma ordem global das operações que preserve a ordem de cada processo. Uma escrita terminada antes de uma leitura em outro processo pode ocupar posição posterior nessa ordem. A linearizabilidade acrescenta a restrição de precedência em tempo real (HERLIHY; WING, 1990).

A comunicação entre réplicas impõe um custo às garantias de ordenação. Uma leitura pode precisar aguardar a coordenação necessária para identificar um valor admissível. Quando a aplicação aceita versões temporariamente distintas, pode-se reduzir essa sincronização e adotar modelos de consistência mais fracos (Van Steen; TANENBAUM, 2023, cap. 7).

1.3.3 CONSISTÊNCIA FRACA E CONSISTÊNCIA EVENTUAL

Consistência fraca designa as garantias que dispensam a visibilidade imediata das escritas: depois que um cliente atualiza uma réplica, leituras em outras réplicas podem devolver o valor anterior até que a atualização se propague (VOGELS, 2009). O termo abrange modelos distintos, cada um com suas restrições de ordem e visibilidade, e por isso é preciso indicar qual deles um sistema oferece.

Por outro lado, a consistência eventual é o caso mais conhecido, já que, se cessarem as escritas, a propagação e a reconciliação das atualizações levam todas as réplicas ao mesmo valor (VOGELS, 2009). A garantia afirma que a convergência ocorre, mas não quando, e até lá clientes diferentes podem observar versões diferentes. A duração desse intervalo depende da comunicação entre as réplicas, da disponibilidade delas e do mecanismo de reconciliação (VOGELS, 2009).

Considere duas réplicas de um cadastro, inicialmente com o telefone v_0. Uma atualização grava v_1 na primeira réplica; antes de sua propagação, uma consulta à segunda ainda pode retornar v_0. Se não houver novas alterações e a atualização for entregue, ambas passarão a armazenar v_1. Caso duas atualizações concorrentes atribuam valores diferentes, a convergência exige uma regra de reconciliação comum (Van Steen; TANENBAUM, 2023, cap. 7).

Ao alternar entre réplicas, um cliente pode deixar de observar uma atualização que já havia lido, problema tratado por Terry et al. (1994) com garantias de sessão que restringem quais réplicas podem atender suas operações. Entre essas garantias, a leitura das próprias escritas exige que as leituras posteriores reflitam as atualizações realizadas na sessão, enquanto as leituras monotônicas asseguram que consultas sucessivas reflitam um conjunto não decrescente de escritas. Os autores também definem escritas monotônicas e escritas que seguem leituras, preservando, respectivamente, a ordem das atualizações da sessão e a precedência das atualizações observadas pelas leituras das quais uma escrita depende.

Como as réplicas armazenam cópias dos mesmos dados lógicos, sua convergência assegura que elas concordem sobre os valores armazenados (SHAPIRO et al., 2011), enquanto uma operação entre serviços, como um pagamento seguido de uma reserva de estoque, modifica dados distintos cuja relação deve satisfazer as regras de integridade da aplicação. A execução dessas operações exige coordenar os efeitos produzidos por cada serviço e tratar as situações em que apenas parte deles é concluída.

1.3.4 EVOLUÇÃO DOS MODELOS E DESENVOLVIMENTOS RECENTES

Os modelos de ordenação têm raízes no estudo de memória compartilhada e de objetos concorrentes, como mostram os trabalhos de Lamport (1979) e Herlihy e Wing (1990). A expansão de serviços replicados pela Internet deu maior relevância a modelos que admitem versões temporariamente diferentes. Vogels (2009) discute essa escolha no contexto de serviços em grande escala, relacionando a propagação das atualizações às garantias observáveis pelos clientes.

Uma linha posterior de desenvolvimento procura obter convergência com condições matemáticas explícitas. Shapiro et al. (2011) formalizam os tipos de dados replicados

livres de conflitos, ou *Conflict-free Replicated Data Types* (CRDTs). Sob as condições de cada construção, réplicas que incorporaram as mesmas atualizações alcançam estados equivalentes, mesmo quando essas atualizações foram recebidas em ordens diferentes. Um contador cujos incrementos podem ser combinados de forma compatível é um exemplo dessa abordagem.

CRDTs permitem atualizar réplicas localmente por meio de operações e regras de combinação que asseguram a convergência, oferecendo consistência eventual forte ao garantir estados equivalentes às réplicas que incorporaram o mesmo conjunto de atualizações. Essa garantia difere da linearizabilidade, que exige uma ordenação das operações compatível com sua precedência em tempo real (SHAPIRO et al., 2011; HERLIHY; WING, 1990), e permite reduzir a necessidade de coordenação entre réplicas, como discutem Van Steen e Tanenbaum (2023, p. 408–409).

1.4 Sagas e transações compensatórias

1.4.1 ORIGEM E DEFINIÇÃO

Garcia-Molina e Salem (1987) introduziram Sagas para tratar transações de longa duração. Uma transação pode permanecer ativa por muito tempo porque acessa muitos objetos, realiza cálculos extensos ou aguarda entradas do usuário. Quando mantém recursos ocupados durante todo esse intervalo, ela pode atrasar outras transações e ampliar o custo de uma eventual falha. Os autores investigam situações em que a atividade pode ser dividida em etapas que liberam recursos antes da conclusão do conjunto.

Uma Saga é uma transação de longa duração decomponível em uma sequência de transações menores, cujas execuções podem ser intercaladas com outras transações. Cada etapa preserva as condições de consistência que lhe cabem e confirma seus próprios efeitos. As etapas permanecem vinculadas pela obrigação de tratar execuções parciais. O sistema deve concluir a atividade ou executar ações compensatórias que tratem os efeitos já produzidos (GARCIA-MOLINA; SALEM, 1987).

No modelo sequencial, seja $S = \langle T_1, T_2, \ldots, T_n \rangle$ uma Saga. Para cada etapa cujo efeito precisa ser desfeito em caso de interrupção posterior, define-se uma transação compensatória C_i. O resultado esperado é a execução completa

$$T_1, T_2, \ldots, T_n$$

ou, se a execução for interrompida depois de T_k e antes de confirmar a próxima etapa, uma sequência de compensação

$$T_1, T_2, \ldots, T_k, C_k, C_{k-1}, \ldots, C_1, \qquad 1 \leq k < n.$$

A sequência cobre a recuperação após pelo menos uma etapa confirmada (GARCIA-MOLINA; SALEM, 1987). Para $k = 0$, a falha ocorre antes de confirmar T_1: o mecanismo local desfaz essa tentativa e a sequência de compensações é vazia. O caso $k = n$ corresponde à conclusão da Saga. As compensações tratam apenas os efeitos das etapas previamente confirmadas.

1.4.2 SEMÂNTICA DA COMPENSAÇÃO E LIMITES DO ISOLAMENTO

Uma compensação é uma nova transação que corrige o efeito de outra segundo a semântica da aplicação. Ela não equivale à restauração indiscriminada de uma cópia antiga do banco. Entre a execução de T_i e sua compensação C_i, outras transações podem ter realizado alterações legítimas nos mesmos dados. Recuperar todo o estado anterior apagaria essas alterações. A compensação precisa identificar o efeito que deve corrigir e preservar os demais (GARCIA-MOLINA; SALEM, 1987).

Para ilustrar a compensação, Garcia-Molina e Salem (1987) recorrem a uma viagem com vários voos, em que cada trecho é reservado por uma transação própria. Se a reserva de um trecho falha, as reservas anteriores são canceladas. Cancelar a reserva de um passageiro corrige o efeito da etapa e preserva as reservas feitas por outros passageiros no intervalo, o que não ocorreria ao restaurar o estado anterior do banco.

Os efeitos de uma etapa confirmada ficam disponíveis antes do término da Saga, e outra transação pode utilizá-los e concluir sua execução antes de uma compensação. Como a compensação é uma nova transação, ela não desfaz o que essas transações já observaram, e a aplicação precisa admitir essa interação entre execuções.

Em uma sequência de três reservas, por exemplo, podem ser executadas T_1 e T_2, enquanto T_3 é rejeitada. A recuperação executa C_2 e C_1. Se houver uma falha temporária durante essa recuperação, a compensação pendente precisa ser retomada para concluir a recuperação. As ações compensatórias devem ser viáveis no domínio considerado, ou seja, efeitos externos irreversíveis exigem regras específicas de tratamento (GARCIA-MOLINA; SALEM, 1987; RICHARDSON, 2018).

1.4.3 DESENVOLVIMENTO DO CONCEITO EM MICROSSERVIÇOS

Em microsserviços, a Saga responde a um problema criado pela separação dos dados: como cada serviço é dono de seu banco, uma operação que envolve vários serviços não cabe em uma única transação local. Richardson (2018, cap. 4) propõe executá-la como uma sequência de transações locais, uma por serviço, coordenadas por mensagens. Cada serviço confirma sua parte sem esperar os demais e, se uma etapa posterior falhar, as etapas já confirmadas são desfeitas por compensações.

A coordenação pode assumir duas formas gerais. Na coreografia, a conclusão de

uma etapa produz um evento ao qual outros participantes reagem, e as decisões ficam distribuídas entre eles. Na orquestração, um coordenador acompanha o progresso e solicita as ações seguintes aos participantes. A orquestração pode utilizar comunicação assíncrona entre coordenador e participantes. Em ambas, a recuperação depende das transações locais e das compensações previstas (RICHARDSON, 2018, seção 4.2).

Plataformas de execução durável de fluxos oferecem suporte à implementação de Sagas. No Temporal, cada etapa é uma transação local acompanhada de uma compensação registrada pelo desenvolvedor, e a plataforma garante a execução das compensações mesmo após falhas dos processos de trabalho (Temporal Technologies, s.d.). A plataforma organiza a execução, mas a definição da ação compensatória continua dependente do domínio. Esse tipo de plataforma também precisa tratar a evolução do código: o Dapr 1.17 introduziu o versionamento de fluxos, que permite alterar a definição de um fluxo sem interromper as instâncias em andamento (Dapr Project Maintainers, 2026).

Uma Saga distribuída depende de mensagens para comunicar conclusões, solicitar ações e iniciar compensações. A coordenação precisa lidar com o intervalo entre a publicação da informação e seu processamento pelo destinatário. A comunicação orientada a mensagens fornece os mecanismos de transporte e seleção utilizados nessas interações (RICHARDSON, 2018, cap. 4).

1.5 Comunicação orientada a mensagens e publicação e assinatura

1.5.1 MENSAGENS E DESACOPLAMENTO ENTRE PROCESSOS

Na comunicação orientada a mensagens, processos trocam unidades de informação por operações de envio e recebimento. Uma mensagem contém dados que o receptor interpreta segundo um formato e um significado acordados. O envio e a execução da ação correspondente são acontecimentos distintos: entregar uma mensagem ao mecanismo de comunicação não significa que o destinatário já tenha processado seu conteúdo (Van Steen; TANENBAUM, 2023, cap. 4).

Van Steen e Tanenbaum (2023, seções 4.1 e 4.3) distinguem comunicação transiente e persistente. Na primeira, a conservação da mensagem depende das condições de funcionamento dos processos e da infraestrutura durante a comunicação. Na segunda, a infraestrutura armazena a mensagem para permitir sua entrega posterior, inclusive quando o destinatário não está ativo no momento do envio. A comunicação também pode ser síncrona ou assíncrona, conforme o ponto em que o remetente pode continuar sua execução. Persistência e sincronização descrevem dimensões diferentes.

Um *middleware* orientado a mensagens oferece serviços intermediários de comunicação, como armazenamento e encaminhamento. Com filas persistentes, remetente e

destinatário podem executar em momentos distintos: o remetente deposita a mensagem, e o destinatário a retira posteriormente. A infraestrutura assume a responsabilidade de manter e transportar a informação segundo suas garantias, enquanto o processamento do conteúdo continua pertencendo à aplicação receptora (Van Steen; TANENBAUM, 2023, p. 220–223).

1.5.2 MODELO DE PUBLICAÇÃO E ASSINATURA

No modelo de publicação e assinatura, ou *publish-subscribe*, um publicador divulga notificações de eventos e os assinantes registram interesse em determinadas notificações. Um serviço de disseminação relaciona as publicações às assinaturas e as encaminha aos interessados. Um evento representa uma ocorrência, enquanto sua notificação é a informação comunicada sobre essa ocorrência (EUGSTER et al., 2003).

Sejam e uma notificação e s uma assinatura. O predicado $M(s, e)$ é verdadeiro quando os atributos de e satisfazem o interesse registrado em s. O serviço de disseminação seleciona as assinaturas correspondentes e encaminha a notificação segundo as garantias de entrega oferecidas. Cada assinante declara seu interesse, e o publicador permanece independente da lista de destinatários (Van Steen; TANENBAUM, 2023, p. 71–73).

Eugster et al. (2003) sistematizam três dimensões de desacoplamento. O desacoplamento espacial permite que publicadores e assinantes interajam sem referências diretas uns aos outros. O temporal permite que participem em momentos distintos, quando há armazenamento das notificações. O desacoplamento de sincronização permite produzir e receber notificações sem organizar a interação como uma chamada bloqueante entre as aplicações.

A entrega de notificações a assinantes desconectados depende de um serviço persistente que as armazene até que possam ser recebidas, respeitando sua política de retenção, enquanto um serviço sem persistência exige que o assinante esteja conectado no momento da publicação (Van Steen; TANENBAUM, 2023, p. 69–70).

Na modalidade baseada em tópicos, os eventos são classificados por nomes ou categorias que os assinantes selecionam conforme seus interesses, enquanto a modalidade baseada em conteúdo permite definir condições sobre os atributos das notificações. Assim, uma assinatura por tópico pode abranger todas as medições de uma sala, enquanto uma assinatura por conteúdo pode restringir o recebimento às medições dessa sala cuja temperatura ultrapasse determinado limite, oferecendo maior expressividade à seleção ao exigir a avaliação dessas condições sobre as publicações (Van Steen; TANENBAUM, 2023, p. 71–73).

1.5.3 EVOLUÇÃO, EXTENSÕES E EXEMPLOS

Em 1993, Oki et al. (1993) apresentaram o *Information Bus*, uma arquitetura para integrar aplicações e permitir sua evolução contínua. A comunicação anônima por assuntos permitia publicar informação sem conhecer os consumidores. Em 2003, Eugster et al. (2003) sistematizaram as variantes por tópico, conteúdo e tipo, além das dimensões de desacoplamento do modelo.

Em 2019, a OASIS (2019) publicou o MQTT 5.0, que especifica a comunicação por publicação e assinatura baseada em tópicos e inclui recursos como assinaturas compartilhadas, expiração de mensagens e manutenção do estado da sessão, permitindo limitar o tempo de disponibilidade das mensagens e preservar informações da interação conforme as condições negociadas.

Nas assinaturas independentes, diferentes assinantes recebem as publicações dos tópicos selecionados. Em uma assinatura compartilhada do MQTT 5.0, o serviço encaminha cada publicação a um membro do grupo, distribuindo o processamento (OASIS, 2019, seção 4.8).

Outra linha de desenvolvimento organiza eventos em registros persistentes e particionados. A documentação do Apache Kafka 4.1 descreve replicação das partições, consumo a partir de posições no registro e grupos de consumidores (Apache Software Foundation, 2025a). A mesma versão apresenta, em caráter de prévia, o recurso *Queues for Kafka* (KIP-932), baseado em grupos de compartilhamento, ou *share groups*, que repartem registros com granularidade diferente da atribuição tradicional de partições (Apache Software Foundation, 2025b). A seleção de eventos passa a ser combinada com retenção, reprocessamento e distribuição do trabalho.

Em um sistema de monitoramento de salas, sensores publicam medições de temperatura, fazendo com que uma aplicação assine essas notificações para manter o histórico e outra as utilize para emitir alertas. Desse modo, uma nova aplicação passa a receber os eventos ao registrar sua assinatura, sem alterar os destinatários no código dos sensores e uma assinatura por conteúdo pode restringir o recebimento às medições acima de um limite (Van Steen; TANENBAUM, 2023, p. 71–73).

O desacoplamento não dispensa um acordo sobre as garantias de entrega, logo, se o serviço pode entregar uma notificação mais de uma vez, o assinante precisa tolerar o processamento repetido e, se não preserva a ordem, uma sequência de eventos, como a criação e o cancelamento de um pedido, pode chegar invertida e ser interpretada de forma errada (EUGSTER et al., 2003; Apache Software Foundation, 2025a).

1.6 Declaração sobre o uso de Inteligência Artificial

Declaro que utilizei o ChatGPT, da OpenAI, como apoio à organização, à redação e à revisão deste texto e dos arquivos LaTeX e BibTeX. A definição do tema e o direcionamento do trabalho foram realizados por mim, com orientação do professor, e assumo a responsabilidade pelo conteúdo apresentado.

REFERÊNCIAS

Apache Software Foundation. *Apache Kafka 4.1: Design*. [S.l.], 2025. Disponível em: <https://kafka.apache.org/41/design/design/>. Acesso em: 28 out. 2026. Citado na página 14.

Apache Software Foundation. *Apache Kafka 4.1.0 Release Announcement*. 2025. Disponível em: <https://kafka.apache.org/blog/2025/09/04/apache-kafka-4.1.0-release-announcement/>. Acesso em: 28 out. 2026. Citado na página 14.

BERENSON, H. et al. A critique of ANSI SQL isolation levels. In: *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. ACM, 1995. p. 1–10. Disponível em: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-95-51.pdf>. Acesso em: 28 out. 2026. Citado na página 6.

BERNSTEIN, P. A.; HADZILACOS, V.; GOODMAN, N. *Concurrency Control and Recovery in Database Systems*. Reading: Addison-Wesley, 1987. ISBN 0-201-10715-5. Citado 2 vezes nas páginas 5 e 6.

Dapr Project Maintainers. *Dapr v1.17 is now available*. 2026. Disponível em: <https://blog.dapr.io/posts/2026/02/27/dapr-v1.17-is-now-available/>. Acesso em: 28 out. 2026. Citado na página 12.

ESWARAN, K. P. et al. The notions of consistency and predicate locks in a database system. *Communications of the ACM*, v. 19, n. 11, p. 624–633, 1976. Disponível em: <https://doi.org/10.1145/360363.360369>. Citado 2 vezes nas páginas 5 e 6.

EUGSTER, P. T. et al. The many faces of publish/subscribe. *ACM Computing Surveys*, v. 35, n. 2, p. 114–131, 2003. Disponível em: <https://doi.org/10.1145/857076.857078>. Citado 2 vezes nas páginas 13 e 14.

GARCIA-MOLINA, H.; SALEM, K. Sagas. In: *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*. New York: ACM, 1987. p. 249–259. Disponível em: <https://doi.org/10.1145/38713.38742>. Citado 2 vezes nas páginas 10 e 11.

GRAY, J. Notes on data base operating systems. In: *Operating Systems: An Advanced Course*. Berlin, Heidelberg: Springer, 1978, (Lecture Notes in Computer Science, v. 60). p. 393–481. Disponível em: <https://doi.org/10.1007/3-540-08755-9_9>. Citado 2 vezes nas páginas 6 e 7.

GRAY, J. The transaction concept: Virtues and limitations. In: *Proceedings of the Seventh International Conference on Very Large Data Bases*. IEEE Computer Society, 1981. p. 144–154. Disponível em: <https://www.cs.utexas.edu/~dahlin/Classes/GradOS/papers/Gray81.pdf>. Acesso em: 28 out. 2026. Citado na página 5.

HAERDER, T.; REUTER, A. Principles of transaction-oriented database recovery. *ACM Computing Surveys*, v. 15, n. 4, p. 287–317, 1983. Disponível em: <https://doi.org/10.1145/289.291>. Citado 3 vezes nas páginas 5, 6 e 8.

HERLIHY, M. P.; WING, J. M. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, v. 12, n. 3, p. 463–492, 1990. Disponível em: <https://doi.org/10.1145/78969.78972>. Citado 3 vezes nas páginas 8, 9 e 10.

LAMPORT, L. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28, n. 9, p. 690–691, 1979. Disponível em: <https://lamport.azurewebsites.net/pubs/multi.pdf>. Acesso em: 27 out. 2026. Citado 2 vezes nas páginas 8 e 9.

LAMPSON, B. W.; STURGIS, H. E. *Crash Recovery in a Distributed Data Storage System*. [S.l.], 1979. Relatório não publicado, versão de 1 jun. 1979. Disponível em: <https://bwlampson.site/21-CrashRecovery/WebPage.html>. Acesso em: 28 out. 2026. Citado na página 7.

LEWIS, J.; FOWLER, M. *Microservices*. 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>. Acesso em: 27 out. 2026. Citado 2 vezes nas páginas 3 e 4.

OASIS. *MQTT Version 5.0*. [S.l.], 2019. OASIS Standard, 7 mar. 2019. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/os/mqtt-v5.0-os.html>. Acesso em: 27 out. 2026. Citado na página 14.

OKI, B. et al. The information bus: an architecture for extensible distributed systems. *ACM SIGOPS Operating Systems Review*, v. 27, n. 5, p. 58–68, 1993. Disponível em: <https://doi.org/10.1145/173668.168624>. Citado na página 14.

Overleaf. *Tutorials - Overleaf, Editor LaTeX*. 2025. Disponível em: <https://pt.overleaf.com/learn/latex/Tutorials>. Acesso em: 2 jun. 2025. Citado na página 2.

RICHARDSON, C. *Microservices Patterns*: With examples in Java. Shelter Island: Manning Publications, 2018. ISBN 9781617294549. Citado 3 vezes nas páginas 4, 11 e 12.

SHAPIRO, M. et al. Conflict-free replicated data types. In: *Stabilization, Safety, and Security of Distributed Systems*. Berlin, Heidelberg: Springer, 2011. (Lecture Notes in Computer Science, v. 6976), p. 386–400. Disponível em: <https://doi.org/10.1007/978-3-642-24550-3_29>. Citado 2 vezes nas páginas 9 e 10.

TAFT, R. et al. CockroachDB: The resilient geo-distributed SQL database. In: *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. New York: ACM, 2020. p. 1493–1509. Disponível em: <https://doi.org/10.1145/3318464.3386134>. Citado 2 vezes nas páginas 6 e 7.

TANENBAUM, A. S.; Van Steen, M. *Distributed Systems: Principles and Paradigms*. 2. ed. Pearson Prentice Hall, 2007. Disponível em: <https://www.distributed-systems.net/index.php/books/ds2/>. Acesso em: 28 out. 2026. Citado na página 3.

Temporal Technologies. *Saga Pattern*. s.d. Disponível em: <https://docs.temporal.io/design-patterns/saga-pattern>. Acesso em: 28 out. 2026. Citado na página 12.

TERRY, D. B. et al. Session guarantees for weakly consistent replicated data. In: *Proceedings of the Third International Conference on Parallel and Distributed Information Systems*. IEEE, 1994. p. 140–149. Disponível em: <https://doi.org/10.1109/PDIS.1994.331722>. Citado 2 vezes nas páginas 8 e 9.

Van Steen, M.; TANENBAUM, A. S. *Distributed Systems*. 4. ed. distributed-systems.net, 2023. Versão consultada: 4.03, janeiro de 2025. ISBN 978-90-815406-4-3. Disponível em: <https://www.distributed-systems.net/index.php/books/ds4/>. Acesso em: 27 out. 2026. Citado 11 vezes nas páginas 3, 4, 5, 6, 7, 8, 9, 10, 12, 13 e 14.

VOGELS, W. Eventually consistent. *Communications of the ACM*, v. 52, n. 1, p. 40–44, 2009. Disponível em: <https://doi.org/10.1145/1435417.1435432>. Citado 2 vezes nas páginas 8 e 9.

Documento assinado digitalmente
PAULO RODOLFO DA SILVA LEITE COELHO
Data: 29/10/2026 13:22:51-0300
Verifique em https://validar.iti.gov.br